Ride-Sharing Backend

Step-by-Step Beginner's Guide

Understand the Full Project from Zero

What you'll learn: Microservices · Spring Boot · Kafka · Redis · Stripe · Docker

#	Chapter	What you'll learn
1	What is this project?	The big picture in plain English
2	How the project is organized	Folder structure & service map
3	How a ride works end-to-end	Step-by-step ride flow
4	Understanding each service	What each service does & why
5	Kafka — the message bus	Why we use it & how it works
6	Redis — the fast memory store	Geo matching & surge pricing
7	Payment flow	How Stripe integration works
8	How to run the project	docker-compose up & test APIs
9	Key concepts summarised	Quick-reference cheat sheet

Chapter 1 — What Is This Project?

The Big Picture

This project is a backend system for a ride-sharing app — think Uber or Ola. When a rider opens the app and requests a ride, a LOT of things happen behind the scenes. This project builds all of that backend logic.

■ **ANALOGY** Think of this project as the 'engine room' of an Uber-like app. The mobile app is just the face — this project is everything that happens when you press 'Book Ride'.

What happens when a rider books a ride?

1 **Rider opens app and taps 'Book Ride'**
App sends a request to our backend

2 **Backend finds nearby drivers**
Using GPS coordinates stored in Redis

3 **A driver accepts the ride**
Driver app notifies our backend

4 **Rider is notified**
Push notification sent automatically

5 **Ride starts and ends**
Status tracked in the database

6 **Payment is processed**
Stripe charges the rider's card

7 **Everyone gets notified**
Rider gets receipt, driver gets earnings info

Why is it split into many small services?

Instead of one giant application, this project is split into 7 smaller applications called **microservices**. Each service does one specific job.

■■ **ANALOGY** Imagine a restaurant. Instead of one person cooking, serving, billing, and cleaning — there's a chef, a waiter, a cashier, and a cleaner. Each person is an 'expert' in their role. Microservices work the same way.

Service	What it does
API Gateway	The front door — all requests enter here

User Service	Handles rider registration and login
Driver Service	Manages drivers and their GPS locations
Ride Service	The heart — manages the entire ride lifecycle
Pricing Service	Calculates fares and surge pricing
Payment Service	Charges the rider's card via Stripe
Notification Service	Sends push notifications, SMS, emails

Chapter 2 — How the Project is Organized

Folder Structure

Every service follows the exact same internal structure. Once you understand one, you understand all.

```
ridesharing-backend/
■■■ api-gateway/ ← Routes all traffic, checks JWT tokens
■■■ user-service/ ← Registration, login
■■■ driver-service/ ← Driver management, GPS tracking
■■■ ride-service/ ← Core ride logic
■■■ pricing-service/ ← Fare calculation, surge pricing
■■■ payment-service/ ← Stripe payment processing
■■■ notification-service/ ← Push notifications
■■■ config-server/ ← Shared configuration for all services
■■■ eureka-server/ ← Service directory (who is running where)
■■■ docker-compose.yml ← Run everything with one command
■■■ README.md ← How to run the project
```

Inside every service

Every service (e.g. ride-service) has the same internal package structure:

```
ride-service/src/main/java/com/ridesharing/rideservice/
■■■ controller/ ← REST API endpoints (what URLs the service exposes)
■■■ service/ ← Business logic (the 'brain' — what the app actually does)
■■■ repository/ ← Database operations (save, find, delete rides)
■■■ model/ ← Database tables defined as Java classes
■■■ dto/ ← Data shapes for requests and responses
■■■ event/ ← Kafka message shapes (data sent between services)
■■■ config/ ← Setup code (Kafka, Redis, Security, Error handling)
```

■ **HOW IT FLOWS** When a request comes in: Controller receives it → passes to Service → Service uses Repository → Repository reads/writes the Database. This is called the 'layered architecture' pattern.

What is each layer's job?

Package	Example File	Purpose
controller/	RideController.java	Receives HTTP requests. Calls service. Returns response. No business logic here.
service/	RideService.java	All the business rules live here. 'Can this driver accept this ride?' etc.

repository/	RideRepository.java	Talks to the database. Spring generates the SQL for you automatically.
model/	Ride.java	A Java class that maps to a database table. Each field = a column.
dto/	RideRequest.java	Simple data containers. Used to receive data from client or send data back.
event/	RideEvent.java	Data sent over Kafka to other services. Like a message in a messaging app.
config/	KafkaConfig.java	Wiring and setup. Tell Spring how to connect to Kafka, Redis, etc.

Chapter 3 — How a Ride Works End-to-End

Follow a single ride request through the entire system

This is the most important chapter. Read it slowly. Once you understand this flow, you understand the entire project.

STEP 1

Rider sends a ride request

POST /api/rides/request

The rider's app sends their pickup & drop location to the API Gateway.

API Gateway checks the JWT token (is this a valid logged-in user?).

If valid, it forwards the request to Ride Service.

STEP 2

Ride Service saves the ride

Ride Service creates a new Ride in PostgreSQL with status = REQUESTED.

It also saves a temporary key in Redis: ride:request:{id} with a 60-second timer.

If no driver accepts in 60 seconds, this key disappears automatically.

STEP 3

Ride Service publishes a Kafka event

Ride Service sends a message to Kafka topic: ride.requested

This message contains the ride ID, pickup location, and estimated fare.

Ride Service's job is done — it doesn't call Driver Service directly.

STEP 4

Driver Service receives the Kafka event

Driver Service is always listening to the ride.requested topic.

It queries Redis: 'Find all ONLINE drivers within 5km of the pickup point.'

Redis returns a list of driver IDs sorted by distance (nearest first).

STEP 5

Drivers are notified

Each nearby driver gets a push notification: 'New ride request nearby — Rs.150'

In production this uses Firebase Cloud Messaging (FCM).

The first driver to accept wins.

STEP 6

Driver accepts the ride

Driver calls: POST /api/rides/{id}/accept

Ride Service checks: Does the Redis key still exist? (not expired or already taken?)

If yes → assigns driver, deletes Redis key, updates status = DRIVER_ASSIGNED.

Publishes ride.accepted to Kafka.

STEP 7

Ride starts and ends

Driver calls /start → status = STARTED, publishes ride.started

Driver calls /complete → status = COMPLETED, publishes ride.completed

At each step, Notification Service sends a push to the rider.

STEP 8

Payment is triggered automatically

Payment Service is listening to ride.completed on Kafka.

It creates a Stripe PaymentIntent and charges the rider's card.

On success → publishes payment.success

On failure → publishes payment.failed (rider is notified to retry)

STEP 9

Notifications go out

Notification Service listens to ALL events and sends push notifications.

Rider gets: 'Driver found', 'Ride started', 'Ride completed', 'Payment received'

Driver gets: 'Ride accepted confirmation', 'Ride cancelled' (if applicable)

■ **KEY INSIGHT** Notice that Ride Service never calls Payment Service or Notification Service directly. It just publishes events to Kafka and forgets. The other services react independently. This is called 'event-driven architecture' — it keeps services loosely coupled.

Chapter 4 — Understanding Each Service

API Gateway (Port 8080)

The single entry point for all external requests.

- Every request from the app goes through the API Gateway first.
- It checks the JWT token: 'Is this user logged in?'
- If valid, it extracts the user ID and adds it as a header: X-User-Id
- It then forwards the request to the correct service (e.g. /api/rides/* → Ride Service)
- It also does rate limiting: too many requests from one user = blocked

Key file: JwtAuthFilter.java — the filter that runs on every request

User Service (Port 8081)

Handles everything related to riders — registration, login, profiles.

- POST /api/users/register — creates a new rider account
- POST /api/users/login — checks password, returns a JWT token
- GET /api/users/me — returns the logged-in rider's profile
- Passwords are never stored as plain text — BCrypt hashes them
- JWT token is signed with a secret key and expires in 24 hours

Key file: UserService.java — register() and login() methods, generateToken()

Driver Service (Port 8082)

Manages drivers and tracks their real-time GPS location.

- POST /api/drivers/register — driver creates an account with vehicle details
- POST /api/drivers/online — driver goes online, GPS stored in Redis
- POST /api/drivers/location — driver app calls this every 5 seconds with GPS
- POST /api/drivers/offline — driver goes offline, removed from Redis
- GET /api/drivers/nearby — finds drivers within a radius using Redis GEORADIUS
- Also listens to Kafka: marks driver ON_RIDE when they accept, ONLINE when done

Key file: DriverLocationService.java — updateDriverLocation(), findNearbyDrivers()

Ride Service (Port 8083)

The most important service — manages the entire ride lifecycle.

- POST /api/rides/request — rider requests a ride
- POST /api/rides/{id}/accept — driver accepts a ride
- POST /api/rides/{id}/start — driver starts the trip
- POST /api/rides/{id}/complete — driver ends the trip
- POST /api/rides/{id}/cancel — cancel a ride
- GET /api/rides/my-rides — rider's ride history
- Publishes Kafka events at every state change

Key file: RideService.java — the state machine, Kafka publishing, Redis TTL

Pricing Service (Port 8085)

Calculates fares and manages surge pricing.

- GET /api/pricing/estimate — returns estimated fare for a route
- GET /api/pricing/surge/{zone} — current surge multiplier for a zone
- Base fare: Rs.30 + Rs.12/km + Rs.2/min
- Surge pricing: if many rides requested and few drivers online → price goes up (max 3x)
- Demand & supply counters stored in Redis, recalculated every 30 seconds

Key file: SurgePricingService.java — estimateFare(), recalculateSurge() with @Scheduled

---

Payment Service (Port 8084)

Charges the rider after a ride completes — uses Stripe.

- Listens to ride.completed Kafka event (does NOT receive direct REST calls)
- Creates a Stripe PaymentIntent for the ride fare
- On success → publishes payment.success to Kafka
- On failure → publishes payment.failed, rider is notified to retry
- Circuit Breaker: if Stripe is down, stops trying after 50% failures
- Idempotent: checks DB before charging to avoid double-charging

Key file: PaymentService.java — handleRideCompleted(), processStripePayment()

Notification Service (Port 8086)

Sends push notifications at every stage of the ride.

- Listens to ALL Kafka topics: ride.* and payment.*
- Sends push notifications via Firebase FCM (mobile apps)
- Sends SMS via Twilio for critical alerts
- Completely stateless — no database, just consumes events and sends messages
- If notification fails, it logs the error but doesn't crash other services

Key file: NotificationService.java — handleRideEvents(), handlePaymentEvents()

Chapter 5 — Kafka: The Message Bus

What is Kafka and why do we use it?

Kafka is like a **bulletin board** that services post messages to and read from. Instead of Service A calling Service B directly (which creates tight coupling), Service A posts a message to Kafka and forgets. Service B reads it whenever it's ready.

■ **WHY KAFKA?** Direct call (bad): Ride Service calls Payment Service directly. If Payment Service is down → Ride Service also fails. Kafka (good): Ride Service posts 'ride completed' to Kafka. Payment Service reads it when ready. Ride Service never knows if Payment Service was down.

Kafka Topics in this project

Topic (Channel)	Posted By	Read By	Meaning
ride.requested	Ride Service	Driver Service, Pricing Service	New ride looking for a driver
ride.accepted	Ride Service	Notification Service	Driver accepted the ride
ride.started	Ride Service	Notification Service	Trip is now in progress
ride.completed	Ride Service	Payment Service, Notification Svc	Trip ended, charge the rider
ride.cancelled	Ride Service	Driver Service, Notification Svc	Ride was cancelled
payment.success	Payment Service	Notification Service	Payment went through
payment.failed	Payment Service	Notification Service	Payment failed, notify rider

How to produce (send) a Kafka message

```
// In RideService.java — after saving the ride to DB:
RideEvent event = RideEvent.builder()
.eventType("ride.requested")
.rideId(ride.getId())
.riderId(riderId)
.pickupLat(req.getPickupLat())
.fare(ride.getEstimatedFare())
.build();

// Send to Kafka topic "ride.requested", key = rideId
kafkaTemplate.send("ride.requested", ride.getId(), event);
```

How to consume (read) a Kafka message

```
// In DriverService's RideEventConsumer.java:
@KafkaListener(topics = "ride.requested", groupId = "driver-service")
public void handleRideRequested(RideEvent event) {
// This method is called automatically when a new message arrives
List nearbyDrivers = locationService.findNearbyDrivers(
event.getPickupLat(), event.getPickupLon(), 5.0);
// Notify each driver...
}
```

Chapter 6 — Redis: The Fast Memory Store

What is Redis and why do we use it?

Redis is an extremely fast in-memory database. We use it for things that need to be retrieved in milliseconds and don't need to be stored forever — like driver GPS locations and surge pricing multipliers.

■■ **REDIS VS POSTGRESQL** PostgreSQL is like a filing cabinet — great for permanent records but slow to search. Redis is like a whiteboard — lightning fast but gets wiped when you restart. Use Redis for real-time, temporary data. Use PostgreSQL for permanent records.

How we use Redis in this project

Redis Key	Type	Purpose
drivers:locations	GeoSet	Stores every ONLINE driver's GPS. Used for 'find drivers near me'.
ride:request:{id}	String	Temporary 60-second timer. Auto-deletes if no driver accepts.
surge:zone:{zone}	String	Surge multiplier (1.0 to 3.0). Refreshed every 30 seconds.
demand:zone:{zone}	Counter	How many rides requested in this zone (last 5 min).
supply:zone:{zone}	Counter	How many drivers are online in this zone (last 5 min).

Driver location tracking — how it works

```
// Driver app calls POST /api/drivers/location every 5 seconds.
// This stores their GPS in a Redis GeoSet:
geoOps.add("drivers:locations", new Point(longitude, latitude), driverId);

// When a ride is requested, find all drivers within 5km:
Circle area = new Circle(new Point(pickupLon, pickupLat),
new Distance(5, Metrics.KILOMETERS));
GeoResults results = geoOps.radius("drivers:locations", area, args);
// Returns: [driverId1 (0.3km), driverId2 (1.2km), driverId3 (2.8km)]
```

The 60-second ride expiry — how it works

When a ride is created, a key is saved in Redis with a 60-second TTL (time-to-live). If a driver accepts within 60 seconds, the key is deleted and the ride is assigned. If no driver accepts, the key disappears automatically — the ride is cancelled.

```
// Save ride in Redis with 60 second TTL:
redisTemplate.opsForValue().set("ride:request:" + rideId, riderId, 60,
TimeUnit.SECONDS);

// When driver tries to accept — check if key still exists:
```

```
if (!redisTemplate.hasKey("ride:request:" + rideId)) {
throw new RuntimeException("Ride request has expired — no longer available");
}
```

Chapter 7 — Payment Flow with Stripe

How payment works — step by step

1 **Ride completes**
Ride Service updates ride status to COMPLETED and publishes ride.completed to Kafka with the fare amount.

2 **Payment Service receives the event**
Payment Service's @KafkaListener picks up the ride.completed message automatically.

3 **Idempotency check**
Before charging, Payment Service checks: 'Does a payment record already exist for this rideId?' If yes → skip. This prevents double-charging if Kafka redelivers the message.

4 **Stripe PaymentIntent created**
Payment Service calls Stripe API to create a PaymentIntent (a charge request). Stripe processes the payment against the rider's saved card.

5 **Success or failure**
Success → Payment status = SUCCESS, publishes payment.success to Kafka. Failure → Payment status = FAILED, publishes payment.failed to Kafka.

6 **Rider gets notified**
Notification Service picks up payment.success or payment.failed and sends a push notification to the rider.

What is a Circuit Breaker?

Stripe (or any external service) can go down. Without protection, every ride completion would hang waiting for Stripe, slowing down the entire app. A Circuit Breaker monitors Stripe calls and if too many fail, it 'opens the circuit' — stops calling Stripe completely and uses a fallback instead.

■ **CIRCUIT BREAKER ANALOGY** Like an electrical circuit breaker at home — if too much current flows (too many failures), it trips and disconnects the circuit to protect the house (your app). After 30 seconds it tries again ('half-open') to see if Stripe recovered.

```
// Stripe calls are wrapped with @CircuitBreaker:
@CircuitBreaker(name = "stripe", fallbackMethod = "stripePaymentFallback")
private void processStripePayment(Payment payment) {
// ... call Stripe ...
}

// If Stripe is down, this fallback is called instead:
private void stripePaymentFallback(Payment payment, Throwable t) {
```

```
payment.setStatus(PaymentStatus.FAILED);
payment.setFailureReason("Stripe unavailable - will retry");
paymentRepository.save(payment);
}
```

Chapter 8 — How to Run the Project

Prerequisites

You need these installed on your computer:

- Java 17 (JDK)
- Docker & Docker Compose
- Maven (or use the bundled ./mvnw wrapper)

Step 1 — Start everything with Docker Compose

```
# Navigate to the project folder
cd ridesharing-backend

# This starts: PostgreSQL, Redis, Kafka, Eureka, Zipkin,
# Prometheus, Grafana, and all 7 microservices
docker-compose up -d

# Check everything is running
docker-compose ps
```

Step 2 — Access the dashboards

Dashboard	URL	What to see
Eureka Dashboard	http://localhost:8761	See all registered services
Kafka UI	http://localhost:8090	Browse topics and messages
Zipkin Tracing	http://localhost:9411	See request traces across services
Grafana Metrics	http://localhost:3000	Dashboards (login: admin/admin)
API Gateway	http://localhost:8080	All API calls go through here

Step 3 — Test with these API calls

```
# 1. Register a rider
POST http://localhost:8080/api/users/register
{"email":"rider@test.com","password":"pass123","fullName":"Rahul
Singh","phone":"9876543210"}

# 2. Login and get JWT token
POST http://localhost:8080/api/users/login
{"email":"rider@test.com","password":"pass123"}
→ Response: {"token":"eyJhbGci...","userId":"abc123"}
```

```
# 3. Register a driver
POST http://localhost:8080/api/drivers/register
{"email":"driver@test.com","password":"pass123","fullName":"Ram Kumar",
"vehicleNumber":"MH01AB1234","vehicleType":"SEDAN","phone":"9123456789"}

# 4. Driver goes online (use driver JWT)
POST http://localhost:8080/api/drivers/online
Authorization: Bearer
{"latitude":28.6139,"longitude":77.2090}

# 5. Rider requests a ride (use rider JWT)
POST http://localhost:8080/api/rides/request
Authorization: Bearer
{"pickupLat":28.6139,"pickupLon":77.2090,"dropLat":28.7041,"dropLon":77.1025,
"pickupAddress":"Connaught Place","dropAddress":"Rohini"}

# 6. Get fare estimate
GET http://localhost:8080/api/pricing/estimate?distance=12&zone;=zone_center
```

Chapter 9 — Key Concepts Cheat Sheet

Core Concepts

Term	Simple Explanation
Microservices	Splitting one big app into many small independent services. Each does one job.
API Gateway	The single entry point. All requests enter here first. Like a security guard.
JWT Token	A signed string that proves who you are. Sent in every request header.
Kafka	A message bus. Services post events; other services read them. Enables decoupling.
Topic	A named channel in Kafka (like a WhatsApp group). e.g. 'ride.completed'
Producer	A service that sends messages to Kafka. e.g. Ride Service
Consumer	A service that reads messages from Kafka. e.g. Payment Service
Redis	An in-memory (ultra-fast) database. Used for GPS, caches, timers.
GeoSet	A Redis data structure that stores GPS coordinates. Supports 'find nearby' queries.
TTL	Time-To-Live. A timer on a Redis key. When it expires, the key is deleted automatically.
Circuit Breaker	Stops calling a failing service to prevent cascade failures. Tries again after a timeout.
Saga Pattern	Managing a multi-step process across services using events instead of one big transaction.
Idempotency	Doing the same operation multiple times = same result. Prevents double-charging.
Docker Compose	A tool to run multiple services (Postgres, Redis, Kafka, your app) with one command.
Eureka	A service registry. Services register themselves so others can find them by name.
Surge Pricing	Higher fare when demand > supply. Computed from Redis counters, cached for 30s.
State Machine	A set of allowed states and transitions. e.g. REQUESTED → ACCEPTED → STARTED → COMPLETED

The Request Journey — Quick Reference

```
Rider App → API Gateway (JWT check) → [Service]
↓
PostgreSQL (save state)
↓
Redis (cache / GPS)
↓
Kafka (publish event)
↓
```

```
Driver Service Payment Service Notification Service
(find nearby drivers) (charge rider card) (send push notification)
```

You've got this! ■

Start by running docker-compose up -d, open the Eureka dashboard, and follow one ride request through the logs. Reading code is 10x easier once you've seen it run in front of you.